SymCheck

Anonymous Author(s)

Abstract

ACM Reference Format:
Anonymous Author(s). 2026. SymCheck. In . ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Property-based testing is an approach to testing programs. In this approach, a programmer identifies *properties*, or specifications, of functions or groups of functions, and those properties are then tested on a large number of randomly selected values. Property-based testing was first popularized in the functional programming ecosystem, with the Haskell library QuickCheck [3] being a particular influential work. However, there are now property-based testing libraries for many other languages, both imperative and functional, including libraries for Python [12], Rust [5], **Bill:** C++? Java? 2 or 3 more here maybe

A traditional style of property based test ensures that two expressions return the same result when applied to equal inputs. For example **Bill:** Show something in quickcheck **Bill:** While undoubtedly useful, ... why might we prefer SymCheck? (1) SymCheck works as a specification (2) SymCheck is independent of existing code– I'm thinking one good example here could be that zip/zipWith have similar behavior- but also these might have very similar implementations

In this work, we propose a new embedded Domain Specific Language (eDSL), SymCheck, for the purpose of writing property-based tests. A test for **Bill:** function from QuickCheck example written in SymCheck is **Bill:** shown in a figure. billTalk about example, call back to points (1) and (2) above

Our contributions are as follow:

- **Bill:** Core language for eDSL, formal semantics for both generating and validating eDSL
- **Bill:** Implementation techniques for eDSL in a statically typed language, Haskell. Discussion of applicability to imperative/dynamically typed languages
- **Bill:** evaluation/case studies

2 Motivating Example

3 Background

4 Design

(1) Π is our path, or "context." It keeps track of which labels are taking on which values currently.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

$\tau \to \beta$	Base type
$\mid [\tau]$	List of τ

Figure 1: Types Bill: Not sure where we want to first talk about this

(2) Γ is our environment. It functions as a dictionary that maps labels to values. Since a single variable may take on multiple values in different contexts, the environment really maps labels to dictionaries which map contexts to values.
(3) Δ is our dependency map. Since certain variable are only defined in the context of other variables, we need a map to tell us which other variable must be defined to bring another variable into scope. So the dependency maps labels to lists of labels for this purpose. For instance, consider the symbolic expression $a \to (b \to b)*_a$. Since b is defined in the star expression labelled "a" then b is only defined if a is defined in the current context of an expression. Thus, $\Gamma(\Pi, \Delta)b = [a]$. When using the environment, we assume the dependency map is precomputed. We also assume the dependency map is consulted before looking up a variables specific value in the environment.
(4) $\Gamma[(\Pi, E) \to \ell]x$ means "Insert the key-value pair (ℓ, x) into E." if E is Γ then we assume that this operation takes the current context and dependency map into account
(5) $\Gamma(\Pi, E)\ell$ means "get the value for key x in E." If E is Γ then we assume that this operation takes the current context and dependency map into account.
(6) $E \in \Gamma\ell$ means "check if ℓ is in the key set of E"
(7) $x : T$ means "x is of type T" where T is either a base type β, an arbitrary type τ, or. A base type is a type that is not a list, an arbitrary type either a base type or a list of some arbitrary type. TODO, this isn't quite true I don't think... it really means "surface expression x will generate values of type T." maybe need new symbol for this?
(8) $e \Downarrow x$ means "symbolic expression e generates value x"
(9) $x \longrightarrow_2 y$ means "surface expression x translates to core expression y"
(10) $xs + +ys$ means "append the list ys to the list xs"
(11) $x :: xs$ means "prepend the element x to the list xs"
(12) $x\ fresh$ means that value x is a random value that will never be used again in any other expression
(13) $x\ arbitrary$ means that value x is a random value that may be used again in subsequent expressions.

4.1 The Surface Language

While convenient for generating and checking values, the core language is quite small and awkward to write examples in. Hence, we define a larger, more feature rich surface language which we

$s \rightarrow [s]$	List
$\mid s\ '\mid'\ s$	Union
$\mid s\ '\mid\{\ell\}'\ s$	Labeled Union
$\mid s*$	Kleene Closure
$\mid s * \{\ell\}$	Labeled Kleene Closure
$\mid s\ s$	Sequencing
$\mid \ell \mapsto s$	Binding
$\mid (\ell \Longleftarrow b) \mapsto s$	Constrained Binding
$\mid \ell$	Label
$\mid .$	Any Value
$\mid (. \Longleftarrow b)$	Constrained Value
$\mid\ < x >$	Program Code
$\mid [\ ]$	Empty List

Figure 2: The Surface Language as a Context Free Grammar

then translate into the core language. Figure 2 shows the grammar of this core language.

Features Many features of the core language closely resemble the core language– we have labels, as well as labeled unions, labeled kleene closures, and labeled bindings, for instance. However, we introduce a number of additional features. We also have unlabeled variants on unions, closures, and bindings, in the form of the unlabeled union, unlabled kleene closure, and unlabeled dot (.) (which represents any value.) Unlike in the core language, labels in the surface language may *optionally* include a predicate, but are now required to have one. Finally, and perhaps most notably, whitespace based sequencing (evocative of traditional regular expressions) to represent lists, rather than the explicit :> and : ++ of the core language.

Translation The surface-to-core translation is split into two phases. The first phase rewrites the surface language to a subset of the surface language, in which we have only labeled features (no unlabeled features), and all labels have associated predicates. The second phase rewrites this subset of the surface language to the core language.

Phase 1 We define the first translation phase as a relation $\longrightarrow_1$, where $s_1 \longrightarrow_2 s_1$ denotes that the expression s is translated to the core language expression s_2. Figure 4a shows **Bill:** a selection of? the rules used in the phase 1 translation. The dot, and unlabeled variants of union and kleene closure, are essentially syntactic sugar for the labeled variant of each construct, and are translated as such in phase 1. For instance, as shown by the rule DotNoPredTrans. the dot operator ., is translated into introducing and then matching on a fresh unconstrained label, $(\ell' \Longleftarrow True) \mapsto \ell'$. Similarly, fresh labels are introduced and bound to each unlabeled union and kleene closure by rules UnlabelledUnionTrans and UnlabelledStarBaseTrans, respectively.

Phase 1 also translates labels that do not have predicates into labels with predicates via the AssignmentNoPredTrans and DotNoPredTrans rules. It does this by simply setting such predicates to $True$, which is trivially satisfied by any value. **Bill:** Not technically a predicate

Phase 2 Phase 2 translates the surface language to the core language, $s \longrightarrow_2 c$. In doing so, it resolves arguably the largest semantic deviation between the surface and core language: the two languages representations of lists. In the core language, a single element is concatenated to a list by :>, and two lists may be concatenated into a single list by : ++. The surface language does away with these operators, using whitespace to denote sequencing– `5 6 7` would represent the list $[5, 6, 7]$, for example. Square brackets are used to represent nested lists– for instance `[5 6] [7] [8 9]` represents the nested list $[5, 6], [7], [8, 9]$. **Bill:** Maybe elaborate on motivation/design choices somewhere

The rules ConcatBaseTrans, ConcatConsTrans,ConcatSnocTrans, and ConcatTrans implement this. They rely on examining the type of the left and right expressions $s_1\ s_2$ in a sequence. In the case that both s_1 and s_2 both have type $[\tau]$, ConcatTrans concatenates the two lists into a single list with type $[\tau]$. If s_1 has type τ and s_2 has type $[\tau]$, then ConcatConsTrans says that s_1 is cons-ed onto the list s_2. Similarly, if s_1 has type $[\tau]$, and s_2 has type τ, then ConcatSnocTrans snoc's s_2 on the end of s_1. Finally, ConcatBaseTrans handles when *both* s_1 and s_2 are the same non-list type β by combining them into the list s_1 :> $[s_2]$. Any other combination of types (for example, if s_1 has type τ and s_2 has type $[[\tau]]$) is rejected with a compile-time **Bill:** I think? Evan, is it compile time? error message. **Bill:** More motivation of design choices?

This difference in the representation of lists does have some slight impact on the translation of the kleene star, in particular. In the core language, we require that the type of the repeated expression be a list. In the surface language, we allow applying the kleene star to either a list *or* a base type, i.e. $5*$ and $[5]*$ would both be lists of 0 or more 5s. To support this, we have two rules to support translation of kleene stars, StarTrans and StarBaseTrans. StarTrans translates stars $x * \{\ell\}$ where x is a list type, by simply translating $x \longrightarrow_2 x'$ and then returning $x'*_\ell$. StarBaseTrans translates stars $x * \{\ell\}$ where x is a base type, and works by translating of x to x', and then wrapping x' in a list in the star, $[x']*_\ell$.

Bill: ...

Bill: Incorporate The surface language is the language that the user interacts with. Their expressions are defined in terms of the surface language so they can be translated to the core language. In this specification, ℓ is a label (i.e a string), x is any arbitrary expression, and b is some expression that evaluates to a boolean. The b expression exists so that the user can constrain the domains of variables so that the generation process isn't completely random. If b is just "$True$" then SymCheck will use the first value it generates for the label. It is possible, however, to include a predicate here which can include surface expressions as arguments so that labels can be constrained in terms of other labels. If this is done, it is assumed that the arguments are translated to the core language (see section 4.3) and evaluated (see section 4.7) before being passed to the predicate.

$$
\begin{aligned}
c \rightarrow\ & c' |'_{\ell} c \\
& \mid c *_{\ell} \\
& \mid c :> c \\
& \mid c :++ c \\
& \mid (\ell \Longleftarrow b) \mapsto c \\
& \mid \ell \\
& \mid\ < x > \\
& \mid [\ ]
\end{aligned}
$$

Figure 3: The Core Language as a Context Free Grammar

4.2 The Core Language

While the user works with the surface language, the core language (see figure 3). The core language is depth-indexed, meaning that the depth of the core AST is part of its type. The depth-index indicates the nested depth of the list it will generate. A depth of 0 indicates that the result will be a singleton not a list. We use this feature to ensure that all expressions are well formed before runtime. Furthermore, we can put constraints on what depths are allowed for each constructor. For example, the : ++ operator is for concatenation which requires 2 lists. So we can ensure that its two arguments have the same depth, and that it is non zero.

4.3 Translation Rules

4.4 Dependency Map Computation

the dependency map function is a bit complex and I'm not sure if we need rules for it anyways or if its sufficient to just describe the algorithm. Since we build the dependency map from a whole series of expressions representing a function I designed it so that it could be a fold function (i.e its type is dmap -> ast n a -> dmap)

(1) TODO: rules go here

4.5 Scope Checking Rules

The scope checking rules aim to check if every label used in a tree is used in an appropriate place which really boils down to two rules

(1) No label is used without first being declared
(2) No label is used unless the dependency map allows it

(1) TODO: add rules

4.6 Well Formedness Rules

The well formedness rules aim to make sure that the depths of each expression and subexpression match the rules of our Ast n a GADT (this may already be done in the translation rules?)

(1) TODO: add rules

4.7 Generation Rules

We now consider generating random input values from symbolic expressions. To do this, we define a relation $(\Gamma, \Pi)\ \ell \Downarrow x,\ \Gamma$. Π is the *path* and Γ is the *environment*; we elaborate on each of these in the following. Understanding the path requires noting that, in generating lists (or, when we come to it later in Section **Bill:** whatever, validating lists) we may repeatedly use an expression via a kleene closure, i.e. given $r*_{\ell}$, the expression r may be repeated an arbitrary number of times. The path is a set of label-to-int mappings that tracks this repetition– on first entering a kleene closure $r*_{\ell}$, $\ell \rightarrow 1$ is entered into the path. On each successive n^{th} iteration, this mapping is updated to n.

The environment tracks how generation is assigning labels to values via bindings, $(lab \Longleftarrow p(args, \ell)) \mapsto e$ however note that the same label may be re-assigned different values in each iteration of a (possibly nested) kleene closure, thus, the environment is really a mapping of (label, path) pairs to values.

Example 4.1. Consider the expression:

$$(n \Longleftarrow True) \mapsto ((x \Longleftarrow True) \mapsto x)*_n$$

One list that could be generated from this would be:

$$[8, 4, 17]$$

The corresponding environment would be:

$$
\begin{aligned}
\{ & (n, \{\}) \rightarrow 3, \\
& (x, \{n \rightarrow 1\}) \rightarrow 8, \\
& (x, \{n \rightarrow 2\}) \rightarrow 4, \\
& (x, \{n \rightarrow 3\}) \rightarrow 17 \\
\} &
\end{aligned}
$$

n occurs outside of all kleene closures, and thus has an empty context. x is chosen to be 8, 4, and 17, respectively, on the 1st, 2nd and 3rd iterations of the unrolling of the kleene closure indexed by n, and thus has a separate value for each of these three contexts.

Figure 5 shows the generation rules.

Lists Constructors Empty, Cons, and Concat structurally match on a symbolic list, recursively generating a concrete list's subcomponents, and then appropriately composing those subcomponents together. Empty is the simplest of these, simply saying that an empty list can only generate an empty list $((\Gamma, \Pi)\ [\ ] \Downarrow [\ ],\ \Gamma.)$ Cons generates a single element for the head of the list and generates the tail of the list, and composes them with :>. Finally, Concat generates two separate lists, and concatenates them together with : ++. **Bill:** Evan, it looks like the recursive applications of $\Downarrow$ on the top of Concat and Cons are a bit malformed– can you please look at those? Also, we need to explain in this paragraph why the context is "threaded through" the recursive calls. (Is the environment threaded through? I think we don't need to have it threaded? But explain that also.)

Literals Literal matches on a specific literal– in a similar fashion to Empty, this rule simply straightforwardly says that a literal can only generate exactly that literal.

Assignment and Variables Assign handles binding of a new label, $(\ell \Longleftarrow p(args, \ell)) \mapsto e$. To do this, it repeatedly generates arbitrary values of the appropriate type **Bill:** how do we know appropriate type?, until it finds a value that satisfies the constraint $p(args, \ell)$. Once such a value is found, the label is bound and returned in a new environment, $\Gamma[(\Pi, \Gamma) \rightarrow \ell]x$. Given a label appearing in the expression, Variable then allows looking up that label in the environment, and generating the mapped-to value.

$$\textsc{UnlabelledStarBaseTrans}\ \frac{}{x* \longrightarrow_1 (\ell' \Longleftarrow True) \mapsto x*_{\ell'}}$$

$$\textsc{UnlabelledUnionTrans}\ \frac{\ell'\ fresh}{x \mid y \longrightarrow_1 (\ell' \Longleftarrow True) \mapsto x \mid_{\ell'} y}$$

$$\textsc{DotWithPredTrans}\ \frac{fresh(\ell')}{(. \Longleftarrow p\ args) \longrightarrow_1 (\ell' \Longleftarrow p(args, \ell)) \mapsto \ell'} \quad \textsc{DotNoPredTrans}\ \frac{(\ fresh \ell')}{. \longrightarrow_1 (\ell' \Longleftarrow True) \mapsto \ell'}$$

$$\textsc{AssignmentNoPredTrans}\ \frac{}{\ell \mapsto x \longrightarrow_1 (\ell \Longleftarrow True) \mapsto x}$$

(a) Bill: Translation Rules Phase 1

$$\textsc{AppTrans}\ \frac{args' = \forall arg_i \in args.arg_i \longrightarrow_2 arg_i'}{< f\ \ args > \longrightarrow_2 < f\ \ args' >} \quad \textsc{UnionTrans}\ \frac{x : \tau \quad y : \tau \quad x \longrightarrow_2 x' \quad y \longrightarrow_2 y'}{x | \{\ell\} y \longrightarrow_2 x' \mid_\ell y'}$$

$$\textsc{StarBaseTrans}\ \frac{x : \beta \quad x \longrightarrow_2 x'}{x * \{\ell\} \longrightarrow_2 [x']*_\ell} \quad \textsc{StarTrans}\ \frac{x : [\tau] \quad x \longrightarrow_2 x'}{x * \{\ell\} \longrightarrow_2 x'*_\ell}$$

$$\textsc{ConcatBaseTrans}\ \frac{x : \beta \quad y : \beta \quad x \longrightarrow_2 x' \quad y \longrightarrow_2 y'}{x\ y \longrightarrow_2 x' :> [y']} \quad \textsc{ConcatConsTrans}\ \frac{x : \tau \quad y : [\tau] \quad x \longrightarrow_2 x' \quad y \longrightarrow_2 y'}{x\ y \longrightarrow_2 x' :> y'}$$

$$\textsc{ConcatSnocTrans}\ \frac{x : [\tau] \quad y : \tau \quad x \longrightarrow_2 x' \quad y \longrightarrow_2 y'}{x\ y \longrightarrow_2 x' :++ [y']} \quad \textsc{ConcatTrans}\ \frac{x : [\tau] \quad y : [\tau] \quad x \longrightarrow_2 x' \quad y \longrightarrow_2 y'}{x\ y \longrightarrow_2 x' :++ y'}$$

$$\textsc{AssignmentWithPredTrans}\ \frac{\forall arg \in args, arg \longrightarrow_2 arg' \quad x \longrightarrow_2 x'}{(\ell \Longleftarrow p(args, \ell)) \mapsto x \longrightarrow_2 (\ell \Longleftarrow p(args', \ell)) \mapsto x'}$$

$$\textsc{Variable}\ \frac{}{\ell \longrightarrow_2 \ell}$$

$$\textsc{LitTrans}\ \frac{}{< c > \longrightarrow_2 < c >}$$

$$\textsc{EmptyTrans}\ \frac{}{[\ ] \longrightarrow_2 [\ ]}$$

(b) Bill: Translation Rules Phase 2

Figure 4: Bill: Translation Rules

$$\textsc{Variable}\ \frac{\Gamma(\Pi, \ell) = x}{(\Gamma, \Pi)\ \ell \Downarrow x,\ \Gamma}$$

$$\textsc{Empty}\ \frac{}{(\Gamma, \Pi)\ [\ ] \Downarrow [\ ],\ \Gamma}$$

$$\textsc{UnionLeft}\ \frac{\Gamma(\Pi, \ell)\ \ \mathrm{mod}\ 2 = 0 \quad (\Gamma, \Pi)\ r_1 \Downarrow x,\ \Gamma'}{(\Gamma, \Pi)\ r_1 |_\ell r_2 \Downarrow x,\ \Gamma'} \quad \textsc{UnionRight}\ \frac{\Gamma(\Pi, \ell)\ \ \mathrm{mod}\ 2 = 1 \quad (\Gamma, \Pi)\ r_2 \Downarrow x,\ \Gamma'}{(\Gamma, \Pi)\ r_1 |_\ell r_2 \Downarrow x,\ \Gamma'}$$

$$\textsc{Star}\ \frac{\Gamma(\Pi, \Gamma_0)\ell = n \quad xs = \forall 1 \leq i \leq n.(\Gamma_{i-1}, \Gamma[(\Pi, \Pi) \rightarrow \ell]i)r \Downarrow xs_i, \Gamma_i}{(\Gamma_0, \Pi) r*_\ell \Downarrow concat(xs), \Gamma_n}$$

$$\textsc{Concat}\ \frac{(\Gamma, \Pi)\ r_1 \Downarrow xs,\ \Gamma' \quad r_2 \Downarrow ys,\ \Gamma''}{(\Gamma, \Pi)\ r_1 :++ r_2 \Downarrow xs ++ ys,\ \Gamma''} \quad \textsc{Cons}\ \frac{(\Gamma, \Pi)\ r_1 \Downarrow x,\ \Gamma' \quad \Gamma' r_1 \Downarrow xs,\ \Gamma''}{(\Gamma, \Pi)\ r_1 :> r_2 \Downarrow x : xs,\ \Gamma''}$$

$$\textsc{Literal}\ \frac{}{(\Gamma, \Pi)\ \ < c > \Downarrow c,\ \Gamma}$$

$$\textsc{Apply}\ \frac{(\Gamma, \Pi)\ args \Downarrow args' \Gamma}{(\Gamma, \Pi)\ \ < f\ \ args > \Downarrow f(args'), \Gamma}$$

$$\textsc{Assign}\ \frac{x\ arbitrary \quad p(args, x)}{(\Gamma, \Pi)\ (\ell \Longleftarrow p(args, \ell)) \mapsto e,\ \Gamma[(\Pi, \Gamma) \rightarrow \ell]x}$$

Figure 5: Bill: Generation Rules

Union UnionLeft and UnionRight handle the $\|_\ell$ operator. Note that which rule applies is based on the (previously randomly chosen) value of ℓ, hence getting us an (indirectly) non-deterministic choice.

Kleene Closure Star handles kleene closures, $r*_\ell$. To do this, it looks up $\Gamma(\Pi, \ell) = n$ in the environment, and then recursive applies $\Downarrow$ to generate n lists that match r, i.e. $\forall 1 \leq i \leq n.(\Gamma_{i-1}, \Gamma[(\Pi, \Pi) \rightarrow \ell]i)r \Downarrow xs_i, \Gamma_i$. Note that (1) the environment is threaded through all these calls, to ensure that we record all variable bindings performed within each regular expression and (2) as covered in our description of the path above, this is the one place in the rules where we *modify* the path, by, on each iteration, setting the label ℓ to point to the current iteration count i.

Apply **Bill:** Evan, the rule might need to be adjusted a bit? Maybe you could also take a stab at writing something here?

4.8 Validation Rules

Validation happens differently than generation. For generation, when a variable is declared we generate a random value for it that satisfies its constraint and insert that into our environment. For

validation, we are given a list and a core expression. So when a variable is declared we have to find if there is some value for the label such that the computed output would be generated from the core expression. Because of this we will have to be splitting lists to see if two expressions can fully match each half. We do that using this syntax $x[i..j]$ which is a sublist of the list x containing all the elements from indices i to j (right exclusive) in order; $x[i..]$ is a special version of this that takes all element from i to the end of the list.

5 Implementation

6 Case Studies

7 Related Work

Property-Based Testing In this work, we consider a new approach to property-based testing based on writing function specifications in a *symbolic expressions* eDSL. We explore this direction in the functional programming world, where in particular QuickCheck [3], alongside other approaches such as SmallCheck [13] and Target [14], has led to property-based testing becoming a popular paradigm.

QuickCheck [3] is often credited as popularizing property-based testing and inspiring property-based testing libraries [1, 4, 12, 14]. However, the earliest work on property-based testing [7, 8] was actually on C code in the Unix community, where property based testing was used to check security properties and search for exploits. It introduced a specialized language, TASPEC, for writing testing oracles and specifications, with an eye specifically toward security.

Despite this, the statement that QuickCheck [3], a Haskell property-based testing library, was highly influential on much, if not all, of the work that follows is quite fair. As shown earlier in the paper **Bill:** Where? QuickCheck allows writing specifications over functions using mostly ordinary Haskell code, using typeclass instances to specify how to generate random input values. **Bill:** More on QuickCheck SmallCheck [13] and Target [14] are alternative Haskell property-based testing library. The design of SmallCheck resembles the design of QuickCheck, except that, instead of being tested on randomly chosen inputs, functions are tested on inputs of increasing size. Target tests specifications written as *refinement types* i.e. regular types annotated with decidbale predicates. This refinement types can also be verified with the LiquidHaskell [17] verifier; a major motivation of Target is to act as a complementary tool to LiquidHaskell; for example, by allowing mixing of verification on "easy to verify" code and testing on "hard to verify" code, and by finding counterexample when verification is failing because of an *actual* bug.

Adaptions of the core ideas of QuickCheck exist for a wide variety of other languages, both functional and imperative; as a (much abbreviated) list, we note QuickCheck for Isabelle [2], QuickCh*i*ck for Rocq [4], QuickCheck for Rust [5], Hypothesis for Python [12], PyModel for Python [11], and Jsongen for json web services [1].

Symbolic Expressions Symbolic expressions can be viewed as mini-programs that generate (related) function inputs and outputs. Alternatively, they can be seens as abstractions or generalizations of the inputs themselves. This second view, of the expressions as abstractions, suggests a relation to the manipulation of symbolic program values performed by program analysis tools. The most direct link to our work in this domain is synthesis with abstract examples [6]. The "abstract examples" in question represent strings using a regular expression like language, extended with labeled kleene stars and variables. Unlike SymCheck, these examples are used purely as an *output mechanism*– the synthesizer asks the user if certain abstract example input/output pairs correspond to the function being synthesized, and the user answers yes or no. The abstract example language in [6] is also significantly less sophisticated than ours; kleene stars are only allowed to repeat a single character, variables correspond to a single character, and there are no further features (i.e. no labeled unions or nested lists.)

Other uses of symbolic representations are further removed, but still worth considering. In the world of functional programming, symbolic execution tools such as Rosette [15], G2 [10], SmtEn [16], and CutEr [9] exploit such reasoning to support finding program bugs and (in some cases) verify programs. These tools general reserve symbolic representations for *internal* use, communicating information to the user by generating corresponding concrete values.

8 Conclusion

References

[1] Clara Benac Earle, Lars-Åke Fredlund, Ángel Herranz, and Julio Mariño. 2014. Jsongen: A quickcheck based library for testing json web services. In *Proceedings of the Thirteenth ACM SIGPLAN workshop on Erlang*. 33–41.

[2] Lukas Bulwahn. 2012. The new Quickcheck for Isabelle: Random, exhaustive and symbolic testing under one roof. In *International Conference on Certified Programs and Proofs*. Springer, 92–108.

[3] Koen Claessen and John Hughes. 2000. QuickCheck: a lightweight tool for random testing of Haskell programs. In *Proceedings of the fifth ACM SIGPLAN international conference on Functional programming*. 268–279.

[4] Maxime Dénès, Catalin Hritcu, Leonidas Lampropoulos, Zoe Paraskevopoulou, and Benjamin C Pierce. 2014. QuickChick: Property-based testing for Coq. In *The Coq Workshop*, Vol. 125. 126.

[5] Max Derbenwick. 2025. Property Based Testing in Rust, How is it Used? (2025).

[6] Dana Drachsler-Cohen, Sharon Shoham, and Eran Yahav. 2017. Synthesis with abstract examples. In *International Conference on Computer Aided Verification*. Springer, 254–278.

[7] George Fink and Matt Bishop. 1997. Property-based testing: a new approach to testing for assurance. *ACM SIGSOFT Software Engineering Notes* 22, 4 (1997), 74–80.

[8] George Fink and Karl Levitt. 1994. Property-based testing of privileged programs. In *Tenth Annual Computer Security Applications Conference*. IEEE, 154–163.

[9] Aggelos Giantsios, Nikolaos Papaspyrou, and Konstantinos Sagonas. 2015. Concolic testing for functional languages. In *Proceedings of the 17th International Symposium on Principles and Practice of Declarative Programming*. 137–148.

[10] William T Hallahan, Anton Xue, Maxwell Troy Bland, Ranjit Jhala, and Ruzica Piskac. 2019. Lazy counterfactual symbolic execution. In *Proceedings of the 40th acm sigplan conference on programming language design and implementation*. 411–424.

[11] Jonathan Jacky. 2011. PyModel: Model-based testing in Python.. In *SciPy*. 48–52.

[12] David R MacIver, Zac Hatfield-Dodds, et al. 2019. Hypothesis: A new approach to property-based testing. *Journal of Open Source Software* 4, 43 (2019), 1891.

[13] Colin Runciman, Matthew Naylor, and Fredrik Lindblad. 2008. Smallcheck and lazy smallcheck: automatic exhaustive testing for small values. *Acm sigplan notices* 44, 2 (2008), 37–48.

[14] Eric L Seidel, Niki Vazou, and Ranjit Jhala. 2015. Type targeted testing. In *European Symposium on Programming Languages and Systems*. Springer, 812–836.

[15] Emina Torlak and Rastislav Bodik. 2014. A lightweight symbolic virtual machine for solver-aided host languages. *ACM SIGPLAN Notices* 49, 6 (2014), 530–541.

[16] Richard Uhler and Nirav Dave. 2014. Smten with satisfiability-based search. *ACM SIGPLAN Notices* 49, 10 (2014), 157–176.

[17] Niki Vazou, Eric L Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. 2014. Refinement types for Haskell. In *Proceedings of the 19th ACM SIGPLAN international conference on Functional programming*. 269–282.